ProgMesas — Documentación interna (técnica)

Actualizado el 13/11/2026 · Alan Buzzatto y Valentino Rodriguez

Documentación para quien instala, mantiene o modifica ProgMesas dentro del repositorio CampusT2 (carpeta `progmesas/`). La guía de uso para alumnos, docentes y directivos está en el documento de documentación externa.

1. Arquitectura

CampusT2 es el sitio principal: da el inicio de sesión y el panel de cada rol. ProgMesas es una subpágina, la carpeta `progmesas/`, que usa esa misma sesión. ProgMesas también tiene un repositorio propio (`alan4352/progMesas`); la copia que corre en la escuela es la de CampusT2.

Dos formas de ejecutarse

Modo	Qué corre	Cuándo se usa
PHP (principal)	`api.php` dentro del mismo Apache/nginx del campus	Siempre: XAMPP, servidor Debian y Render
Node (opcional)	`backend/server.js` en el puerto 3000	Solo si alguien lo inicia con `iniciar.bat` / `iniciar.sh`; agrega copias de seguridad automáticas

El campus decide a cuál mandar al usuario en `enlaceProgMesas()` (`campus/includes/progmesas.php`): si hay algo escuchando en `127.0.0.1:3000` usa `PROGMESAS_URL`; si no, `../progmesas`. El frontend es el mismo en los dos modos: siempre llama a `api.php?r=/ruta`, y el servidor Node también acepta esa forma.

Flujo de una petición (modo PHP)

1. El navegador abre `progmesas/` → `index.php`. Si hay sesión del campus y no llegó un `?token=`, redirige a `../campus/programas.php`, que arma el pase y vuelve con el token. Si no, sirve `index.html`.
2. `frontend/common.js` guarda el token y hace las llamadas con `api()`, que agrega `Authorization: Bearer …` y `X-Requested-With: progmesas`.
3. `api.php` identifica al usuario (`obtenerUsuarioActual()`), resuelve la ruta y responde JSON.
4. Los PDF se abren desde Google Drive (vista previa pública) o, si quedaron en el servidor, como archivos estáticos de `backend/uploads/`.

Carpetas y archivos

Ruta	Contenido
`index.php`, `index.html`	Entrada y página principal (consulta y carga)

Ruta	Contenido
`api.php`	Toda la API en modo PHP: rutas, SQLite, lectura de la planilla, almacenamiento de los PDF, ZIP
`backend/drive.php`	Cliente de Google Drive en PHP (API REST v3 y OAuth 2.0, sin librerías)
`backend/autorizar_drive.php`	Autoriza la cuenta de Drive de la escuela desde la línea de comandos
`backend/oauth.json`, `backend/token.json`	Credenciales de la app de Google y permiso de la cuenta (no se suben al repositorio)
`frontend/`	`app.js` (página principal), `dashboard.html` + `dashboard.js` (asesoría), `common.js` (sesión y `api()`), `tema.js` (claro/oscuro), `styles.css`
`backend/data/asignaciones.xlsx`	Planilla oficial de cursos, materias y docentes
`backend/database.sqlite`	Base de datos (programas, vínculos, registro, ajustes)
`backend/uploads/`	PDF que quedaron en el servidor porque Drive no estaba disponible
`backend/*.js`	Servidor Node opcional (`server.js`, `drive.js`, `respaldos.js`, …) y scripts para generar la planilla
`.htaccess`, `backend/.htaccess`	Bloqueos para Apache
`privacidad.html`, `terminos.html`	Textos legales
`LEEME.md`, `README.md`	Guía de uso y notas técnicas

2. Integración con el campus y autenticación

ProgMesas no guarda contraseñas. Reconoce al usuario de dos maneras, en este orden:

a) Pase JWT (principal). `campus/programas.php` llama a `crearPaseProgMesas()`, que firma con HMAC-SHA256 un JWT con `sub` (id del campus), `name`, `email`, `role`, `iat` y `exp` (`PROGMESAS_HORAS`, 8 h por defecto). La clave es `PROGMESAS_SECRETO` de `campus/includes/config.php`. `api.php` lo valida en `verificarTokenJwt()`: firma, algoritmo y vencimiento. Si la clave tiene menos de 32 caracteres o empieza con `CAMBIAR`, no acepta ningún token.

b) Sesión del campus (respaldo). Si no hay token válido, `api.php` carga `campus/includes/sesion.php` (misma cookie segura) y lee `$_SESSION['rol']`, `nombre`, `correo` e `id`. Como la cookie viaja sola, para cualquier método distinto de GET/HEAD exige la cabecera `X-Requested-With: progmesas`, que solo manda `common.js`. Así otra página no puede subir ni borrar con la sesión de alguien (CSRF).

Roles. `mapearRol()` traduce el rol del campus:

Campus	ProgMesas	Uso
`alumno`	`student`	Consulta
`profesor`	`teacher`	Carga y reutilización
`directivo`	`advisor`	Todo, más Dashboard y eliminar
—	`viewer`	Solo lectura del Dashboard. Existe en la API, pero el campus no tiene un rol que lo genere

Los ids de usuario tienen la forma `alumno:<DNI>`, `profesor:<correo>` o `directivo:<correo>`, igual que en el campus.

Enlaces de vuelta. `/config` devuelve `loginUrl` y las direcciones de **Volver al campus** y **Cerrar sesión**, armadas con `urlCampus()` a partir de la carpeta hermana `campus/`. Por eso las dos carpetas tienen que quedar una al lado de la otra.

Servidor Node. Valida el mismo pase con `EXTERNO_JWT_SECRET` de `progmesas/.env`, que tiene que ser igual a `PROGMESAS_SECRETO`.

3. API

Todas las rutas se llaman como `api.php?r=/ruta` y responden JSON. Los errores vuelven como `{"error": "mensaje"}` con código 400, 401 (sin sesión), 403 (sin permiso), 404, 409 o 413. «Pública» quiere decir sin sesión.

Método y ruta	Quién	Qué hace
GET `/config`	Pública	Año lectivo, límites y direcciones del campus
GET `/yo`	Pública	Usuario actual o `null`; registra el acceso del docente
GET `/programas`	Pública	Lista de programas (la asesoría ve campos extra)
GET `/fecha-limite`	Pública	Fecha límite del año
GET `/cursos/zip`	Pública	ZIP con los PDF de un curso y año, bajados de Drive o del servidor (opcional: sólo un tipo)
GET `/docentes/nombres`	teacher, advisor	Nombres de la planilla para vincular
POST `/docentes/vinculo`	teacher	El docente elige su nombre de la planilla
GET `/programas/:id/versiones`	teacher, advisor	Versiones reemplazadas

Método y ruta	Quién	Qué hace
GET `/asignaciones/mias`	teacher, advisor	Materias y cursos, con las del docente primero
POST `/programas/subir`	teacher, advisor	Sube un PDF regular o libre (multipart: `asignacion_id`, `anio`, `tipo`, `pdf`)
POST `/programas/reutilizar`	teacher, advisor	Copia el programa de otro año, del mismo tipo, al actual
GET `/docentes/vinculos`	advisor, viewer	Cuentas y vínculos
POST `/docentes/vinculos/:id/confirmar`	advisor	Asigna o corrige el nombre de una cuenta
DELETE `/docentes/vinculos/:id`	advisor	Desvincula una cuenta
DELETE `/programas/:id`	advisor	Borra el programa y sus versiones, y sus PDF de Drive o del servidor si ningún otro programa los usa
PATCH `/programas/:id/asignacion`	advisor	Asocia un programa viejo a una materia de la planilla
GET `/cobertura`	advisor, viewer	Datos del Dashboard (una materia está completa con el regular o el libre) y estado de Drive
PUT `/fecha-limite`	advisor	Guarda o borra la fecha límite
GET `/registro`	advisor, viewer	Registro de actividad
GET `/programas/faltantes`	advisor, viewer	Pendientes (para el CSV)
POST `/drive/sincronizar`	advisor	Pasa a Drive los PDF que quedaron en el servidor (hasta 50 por vez)

Reglas de la carga

- El año del programa es siempre el lectivo actual (`anioDelPrograma()`); otro año devuelve el error «Sólo se pueden cargar programas del año lectivo actual».
- El título lo arma `tituloAutomatico()`: «Programa de {materia} - {curso} - {año}», o «Programa libre de …» si el tipo es libre (el regular no se aclara en título, etiqueta ni ZIP).

- Máximo `MAX_PDF_MB` (10 MB). Se comprueba que el archivo empiece con `%PDF-`, no solo la extensión.
- Cada materia puede tener un programa regular y uno libre (sin tipo se toma regular; otro valor da error 400). Si ya existe un programa para la misma asignación, año y tipo, el anterior pasa a `programas_versiones` y el nuevo ocupa su lugar.
- Cada acción que cambia datos queda en la tabla `registro` (`registrarAccion()`).

4. Datos

Base SQLite (`backend/database.sqlite`). `db()` crea las tablas que falten la primera vez que se usa; no hace falta importar nada.

Tabla	Para qué	Campos principales
`programas`	Programa vigente de cada materia, curso, año y tipo. La columna tipo vale regular o libre; los programas cargados antes de que existiera quedaron como regulares. Al iniciar se corrigen los títulos que habían quedado como «Programa regular de …»	`id`, `title`, `materia`, `curso`, `year`, `profesor`, `file_url`, `uploaded_by`, `asignacion_id`, `created_at`
`programas_versiones`	Programas reemplazados	`programa_id`, `file_url`, `reemplazado_at`, `reemplazado_por`
`docentes_usuarios`	Vínculo entre cuenta del campus y nombre de la planilla	`usuario_id`, `email`, `docente`, `estado`, `confirmado_por`
`registro`	Auditoría	`fecha`, `usuario_id`, `rol`, `accion`, `detalle` (JSON)
`ajustes`	Clave/valor	Fecha límite por año (`fecha_limite_2026`), último error de Drive (`drive_error`) y último paso a Drive (`drive_sincronizado_at`)

Planilla de asignaciones (`backend/data/asignaciones.xlsx`). Es la lista oficial de qué materias tiene cada curso y quién las dicta; de ahí salen la cobertura y los pendientes. `api.php` la lee con un lector ZIP propio más SimpleXML (no necesita librerías), solo la primera hoja, buscando las columnas por el encabezado:

Columna	Ejemplo	Notas
`ID`	`1-1_ingles`	Si falta, se genera con curso y materia

Columna	Ejemplo	Notas
`Curso`	`1° 1°`	Obligatoria
`Turno`	`Mañana`	
`Materia`	`Inglés`	Obligatoria
`Docentes`	`Acevedo / Grimberg Valeria`	Varios separados por `/`
`Requiere programa`	`SI` / `NO`	`NO` la saca de la cobertura
`Origen`, `Revisar`		Informativas

PDF. `guardarPdf()` los sube a **Google Drive** con `driveSubir()` (`backend/drive.php`): carpetas Año / Curso / Materia dentro de la carpeta raíz de la escuela, con el nombre `año-curso_docente_materia.pdf` (`_libre` si es el libre), y públicos de sólo lectura. En la base quedan `almacenamiento = 'drive'`, `archivo_ref` = id del archivo y `file_url` = su vista previa (`https://drive.google.com/file/d/<id>/preview`).

Si Drive no está configurado o falla, el PDF se guarda en `backend/uploads/` como `<milisegundos>_<curso_materia_tipo>.pdf` (`almacenamiento = 'local'`, `file_url = /uploads/<archivo>`) y, si fue una falla, la respuesta trae un aviso. `pasarADrive()` sube después esos archivos y actualiza los programas y versiones que los usan; la llama el botón **Pasar a Drive ahora** y también `sincronizacionAutomatica()`, al abrir el dashboard, como mucho cada 6 horas (hasta 10 archivos por vez). Un programa reutilizado comparte el archivo del original, y las versiones reemplazadas conservan el suyo hasta que se elimina el programa.

Permiso de Drive. `driveTokenAcceso()` usa el `refresh_token` de `backend/token.json` para pedir un token de acceso a Google y guarda el renovado en el mismo archivo. Si falla, el error queda en `ajustes.drive_error` y el dashboard lo muestra.

5. Instalación y publicación

ProgMesas se instala junto con el campus; no tiene instalación propia. Requisitos de PHP para su parte: 8.1 o superior con `pdo_sqlite`, `xml` (SimpleXML), `zlib`, `mbstring` y `openssl`, y `allow_url_fopen = On` (para hablar con Google Drive).

Configuración que necesita

Dónde	Valor	Para qué
`campus/includes/config.php`	`PROGMESAS_SECRETO`	Clave del pase. Generarla con `php -r "echo bin2hex(random_bytes(32));"`
`campus/includes/config.php`	`PROGMESAS_HORAS`	Duración del pase (8 por defecto)
`campus/includes/config.php`	`PROGMESAS_URL`	Dirección del servidor Node, si se usa

Dónde	Valor	Para qué
`progmesas/backend/oauth.json`	Credenciales de la app de Google	Google Drive (las mismas que usa la versión Node)
`progmesas/backend/token.json`	Permiso de la cuenta de la escuela	Lo crea `php backend/autorizar_drive.php`
Variable `DRIVE_CARPETA_RAIZ` o `PROGMESAS_DRIVE_CARPETA_RAIZ` en `config.php`	Id de la carpeta de Drive	Opcional: por defecto, la carpeta de la escuela
Variables `GOOGLE_OAUTH_JSON`, `GOOGLE_TOKEN_JSON`	Contenido de los dos archivos anteriores	Para Render, donde no se pueden copiar archivos
`progmesas/.env`	`EXTERNO_JWT_SECRET`	Solo para Node: igual a `PROGMESAS_SECRETO`

XAMPP (Windows)

1. Copiar CampusT2 dentro de `htdocs` y crear `config.php` como indica el README del repositorio.
2. Entrar al campus como profesor o directivo y abrir el módulo de programas: la base SQLite y la carpeta `uploads` se crean solas.
3. Apache usa los `.htaccess` de `progmesas/` y `progmesas/backend/`.
4. Google Drive: copiar `oauth.json` a `progmesas/backend/` y, desde la carpeta `progmesas`, ejecutar `C:\xampp\php\php.exe backend\autorizar_drive.php`. Abrir el enlace que muestra, aceptar con la cuenta de la escuela y pegar la dirección a la que redirige.

Servidor de la escuela (Debian + nginx + PHP-FPM, `/var/www/final`)

1. Instalar `php8.4-sqlite3` y `php8.4-xml` además de lo del campus.
2. nginx **no lee los `.htaccess`**: usar `docs/nginx-campus.conf`, que devuelve 404 para todo `progmesas/backend/` salvo los `.pdf` de `uploads/`, y para cualquier `.sqlite`, `.xlsx`, `.env`, etc.
3. El usuario de PHP-FPM (`www-data`) necesita escribir en `progmesas/backend/` (base, sus archivos `-wal`/`-shm` y `uploads/`).
4. Comprobar con `sudo nginx -t` y recargar.
5. Google Drive: copiar `oauth.json` a `progmesas/backend/`, ejecutar `php backend/autorizar_drive.php` desde `/var/www/final/progmesas` y darle a `www-data` permiso de lectura y escritura sobre `backend/token.json`.

Render + Aiven

- Render construye el `Dockerfile` (php:8.3-apache). `docker/entrypoint.sh` crea `progmesas/backend/uploads` y le da permisos a `www-data`.

- `PROGMESAS_SECRETO` la genera Render sola (`render.yaml`); el resto de la configuración llega por variables de entorno (`config.render.php`).
- ProgMesas no usa MySQL: Aiven solo guarda las tablas del campus. La base SQLite vive en el disco del contenedor; los PDF, en Google Drive si se cargan `GOOGLE_OAUTH_JSON` y `GOOGLE_TOKEN_JSON` (si no, también en el disco).
- Cada `git push` a la rama principal vuelve a publicar el sitio.

6. Seguridad

Las reglas generales están en `docs/SEGURIDAD.md` del repositorio. Lo propio de ProgMesas:

- **Permisos por ruta:** cada ruta que no es pública llama a `exigirRol()` al principio. Una ruta nueva tiene que hacer lo mismo.
- **Pase firmado:** sin `PROGMESAS_SECRETO` válido no entra nadie por token. La clave no va en el repositorio ni en el código de `api.php`.
- **Anti-CSRF:** con la sesión de respaldo, todo lo que no sea GET necesita `X-Requested-With: progmesas`. Cualquier llamada nueva desde el frontend tiene que pasar por `api()` de `common.js`.
- **Archivos subidos:** solo PDF (se revisa la firma `%PDF-`), máximo 10 MB, con nombre generado por el servidor. Nunca se usa el nombre que manda el navegador.
- **Datos internos ocultos:** Apache los bloquea con `.htaccess` (`.env`, `package.json`, `.sqlite`, `.js/.json/.xlsx` de `backend/`, y `drive.php` y `autorizar_drive.php`); nginx, con `docs/nginx-campus.conf`. `drive.php` y `autorizar_drive.php` además responden 404 si se abren desde el navegador.
- **Google Drive:** `oauth.json` y `token.json` dan acceso al Drive de la escuela: no se suben al repositorio (`.gitignore`) ni a la imagen de Docker (`.dockerignore`). Los PDF quedan públicos de sólo lectura, igual que en la página.
- **Escape de datos:** el frontend arma el HTML con `esc()` para todo texto que viene de la API.
- **Auditoría:** subir, reemplazar, reutilizar, eliminar, vincular, cambiar la fecha límite y pasar archivos a Drive quedan en `registro` con usuario y fecha.
- **Fuera del repositorio:** `config.php`, `progmesas/.env`, `oauth.json` y `token.json` están en `.gitignore`. Ver la limitación sobre la base y los PDF de ejemplo en la sección 8.

7. Mantenimiento

Cambiar la planilla de asignaciones (cada año o cuando cambia un docente)

1. Editar `backend/data/asignaciones.xlsx` respetando los encabezados de la sección 4. Conviene mantener los `ID` existentes: los programas ya cargados se asocian por ese id.
2. Reemplazar el archivo en el servidor. No hace falta reiniciar: `api.php` lo vuelve a leer en cada petición.
3. Si un programa viejo quedó sin materia, la asesoría lo asocia desde el Dashboard (`PATCH /programas/:id/asignacion`).

Copias de seguridad

Los PDF quedan en Google Drive. Del servidor alcanza con copiar `backend/database.sqlite` (con sus `-wal`/`-shm`, o después de detener PHP), la carpeta `backend/uploads/` (si quedó algún PDF) y `backend/token.json`. Hacerlo antes de cada actualización y, en época de mesas, todas las semanas.

Actualizar el código

- Al copiar una versión nueva al servidor, no pisar `backend/database.sqlite`, `backend/uploads/`, `backend/data/`, `backend/oauth.json`, `backend/token.json` ni `.env`.
- Los cambios hechos en el repositorio `progMesas` se pasan a CampusT2 copiando `api.php`, `index.php`, `index.html`, `frontend/` y los `.php` de `backend/`. Después hay que probar el ingreso desde el campus con los tres roles.
- Actualizar el `README.md` con cada cambio de funciones.

Renovar el permiso de Drive

Si el dashboard dice que el permiso venció, repetir `php backend/autorizar_drive.php`. Para que no venza cada 7 días, la app de Google tiene que estar «En producción» en la pantalla de consentimiento de OAuth.

Cambiar la clave del pase

Cambiar `PROGMESAS_SECRETO` (y `EXTERNO_JWT_SECRET` si se usa Node) invalida todos los pases activos; los usuarios solo tienen que volver a entrar desde el campus.

Diagnóstico rápido

Síntoma	Causa probable
401 en todas las rutas con sesión iniciada	`PROGMESAS_SECRETO` vacío, corto o con `CAMBIAR`, y falta la cabecera `X-Requested-With`
El Dashboard dice «No hay lista de asignaciones»	No existe `backend/data/asignaciones.xlsx`
Error al guardar o «readonly database»	PHP no puede escribir en `backend/`
Error 500 al leer la planilla	Falta la extensión `xml` de PHP
Los PDF dan 404 en nginx	Falta el bloque de `uploads/` de `docs/nginx-campus.conf`
«Google Drive no está configurado»	Falta `backend/oauth.json` o `backend/token.json`
«Google Drive no está disponible: … venció»	Repetir `php backend/autorizar_drive.php`
«PHP no puede conectarse a Google»	Activar `openssl` y `allow_url_fopen` en `php.ini`

8. Limitaciones conocidas y pendientes

Tema	Situación	Qué hacer
Render gratis	El disco del contenedor se borra en cada reinicio: se pierde la base SQLite (los PDF no, si están en Drive)	Para uso real, el servidor de la escuela o un disco persistente
Rol `viewer` (preceptor)	Existe en la API pero ningún rol del campus lo genera	Agregar el rol al campus si se necesita
Base y PDF en el repositorio	`backend/database.sqlite` y cuatro PDF de prueba de `backend/uploads/` están subidos a CampusT2. Hoy solo tienen datos de ejemplo, pero un `git pull` en el servidor puede pisar la base real	Sacarlos del repositorio y agregarlos al `.gitignore`
Sin copias automáticas de la base en PHP	Los PDF quedan en Drive, pero la base SQLite sólo se copia a mano (en Node lo hace `respaldos.js`)	Copias manuales (sección 7) o una tarea programada del servidor
Drive sólo desde el dashboard	Sin un proceso que quede corriendo, los PDF que quedaron en el servidor pasan a Drive cuando la asesoría abre el dashboard o toca el botón	Si hiciera falta, una tarea programada que llame a `/drive/sincronizar`